COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE 21S2
DATE PERFORMED:	July 29. 2026	**DATE SUBMITTED:**	July 29, 2026
NAME:	Mark Joneil Bautista	**INSTRUCTOR:**	Mr. Jimlord Quejado

ACTIVITY NO. 3.1 Hands On Activity: Linked Lists

1. PROCEDURE OUTPUT

Table 3-1

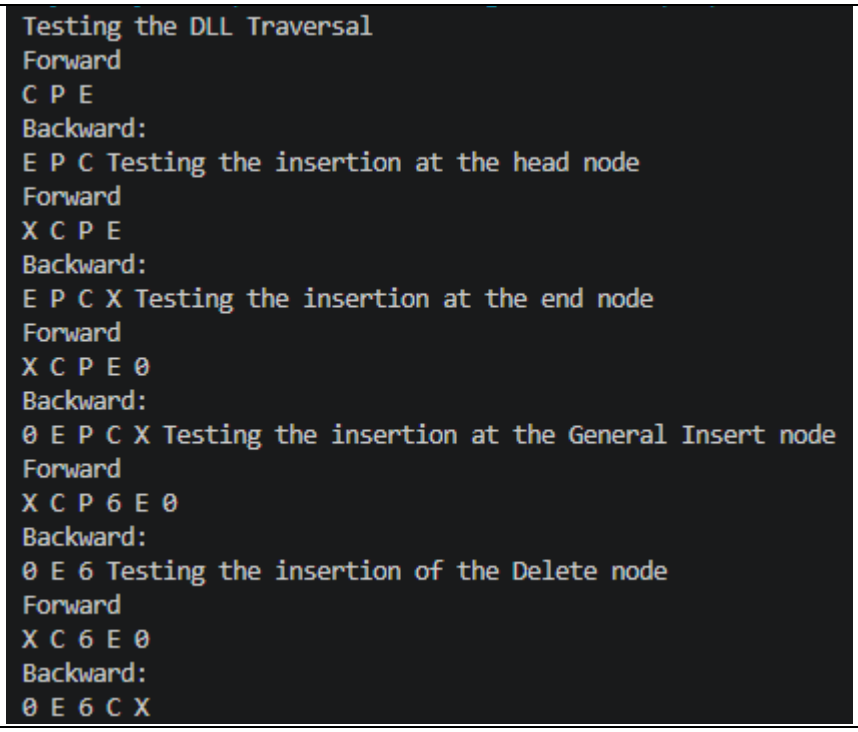
Screenshot	
Discussion	The output confirms the correctness of the doubly linked list by showing that forward and backward traversals remain exact mirrors of each other throughout every operation including head, tail and general insertions as well as node deletion. This symmetry validates that all next and prev pointers are properly updated and linked in both directions without losing structural continuity or leaking memory.

Table 3-2

Operation	Screenshot

Traversal

```
22  template <typename T>
23  void dllTraverse(DoubleList<T>* currentNode){
24      DoubleList<T> *tail;
25
26      if(currentNode == nullptr){
27          std::cout<<"The list is empty"<<std::endl;
28          return;
29      }
30
31      std::cout<< "Forward \n";
32
33      while (currentNode != nullptr) {
34          std::cout<<currentNode->data<<" ";
35          tail = currentNode;
36          currentNode = currentNode->next;
37      }
38      std::cout<<std::endl;
39
40      std::cout<< "Backward: \n";
41
42      while(tail != nullptr){
43          std::cout<< tail-> data<<" ";
44          tail = tail-> prev;
45      }
46
47
48  }
49
```

Insertion at head

```
50    template <typename T>
51    void dllInsertHead(T newData, DoubleList<T>** curre
52        DoubleList<T>* newNode = CreateNewNode(newData)
53        newNode->next = *currentHead;
54
55        (*currentHead)->prev = newNode;
56        *currentHead = newNode;
57    }
58    template <typename T>
59    void dllInsertEnd(T newData, DoubleList<T>* current
60        DoubleList<T>* newNode = CreateNewNode(newData)
61
62        while(currentHead->next != nullptr){
63            currentHead = currentHead->next;
64        }
65        currentHead->next = newNode;
66
67        newNode->prev = currentHead;
68    }
69
```

Insertion at any part of the list

```
template <typename T>
void dllGenInsert(T newData, DoubleList<T>* currentHead){
        if(currentHead->prev == nullptr){
            std::cout<<"Previous Value Cannot be Null"<<std::endl;
            return;
        }
    DoubleList<T>* newNode = CreateNewNode(newData);
    newNode->next = currentHead->next;
    currentHead-> next -> prev = newNode;
    currentHead->next = newNode;
}
```

Insertion at the end

```
57  }
58  template <typename T>
59  void dllInsertEnd(T newData, DoubleList<T>* current
60      DoubleList<T>* newNode = CreateNewNode(newData)
61
62      while(currentHead->next != nullptr){
63          currentHead = currentHead->next;
64      }
65      currentHead->next = newNode;
66
67      newNode->prev = currentHead;
68  }
69
```

Deletion of a node

```
82  template <typename T>
83  void dllDelNode(T findData, DoubleList<T>** curre
84      if(currentHead == nullptr || *currentHead ==
85
86      DoubleList<T>* currNode = *currentHead;
87
88      while (currNode != nullptr && currNode->data
89          currNode = currNode->next;
90      }
91
92      if (currNode == nullptr) return;
93
94      if (currNode == *currentHead) {
95          *currentHead = currNode->next;
96      }
97
98      if (currNode->prev != nullptr) {
99          currNode->prev->next = currNode->next;
100     }
101
102     if (currNode->next != nullptr) {
103         currNode->next->prev = currNode->prev;
104     }
105
106     delete currNode;
107 }
```

A	Source Code	

```
22  template <typename T>
23  void dllTraverse(DoubleList<T>* currentNode){
24      DoubleList<T> *tail;
25
26      if(currentNode == nullptr){
27          std::cout<<"The list is empty"<<std::endl;
28          return;
29      }
30
31      std::cout<< "Forward \n";
32
33      while (currentNode != nullptr) {
34          std::cout<<currentNode->data<<" ";
35          tail = currentNode;
36          currentNode = currentNode->next;
37      }
38      std::cout<<std::endl;
39
40      std::cout<< "Backward: \n";
41
42      while(tail != nullptr){
43          std::cout<< tail-> data<<" ";
44          tail = tail-> prev;
45      }
46
47
48  }
49
```

	Console	

```
Testing the DLL Traversal
Forward
C P E
Backward:
E P C Testing the insertion
```

B

Source Code

```
50  template <typename T>
51  void dllInsertHead(T newData, DoubleList<T>** currentHead){
52      DoubleList<T>* newNode = CreateNewNode(newData);
53      newNode->next = *currentHead;
54
55      (*currentHead)->prev = newNode;
56      *currentHead = newNode;
57  }
58  template <typename T>
59  void dllInsertEnd(T newData, DoubleList<T>* currentHead){
60      DoubleList<T>* newNode = CreateNewNode(newData);
61
62      while(currentHead->next != nullptr){
63          currentHead = currentHead->next;
64      }
65      currentHead->next = newNode;
66
67      newNode->prev = currentHead;
68  }
69
```

Console

```
Forward
X C P E
Backward
E P C X
```

C

Source Code

```
template <typename T>
void dllGenInsert(T newData, DoubleList<T>* currentHead){
        if(currentHead->prev == nullptr){
            std::cout<<"Previous Value Cannot be Null"<<std::endl;
            return;
        }
    DoubleList<T>* newNode = CreateNewNode(newData);
    newNode->next = currentHead->next;
    currentHead-> next -> prev = newNode;
    currentHead->next = newNode;
}
```

Console

```
Forward
X C P 6 E 0
Backward:
0 E 6 Testi
```

D

Source Code

```
57  }
58  template <typename T>
59  void dllInsertEnd(T newData, DoubleList<T>* currentHead){
60      DoubleList<T>* newNode = CreateNewNode(newData);
61
62      while(currentHead->next != nullptr){
63          currentHead = currentHead->next;
64      }
65      currentHead->next = newNode;
66
67      newNode->prev = currentHead;
68  }
69
```

Console

```
Forward
X C P E 0
Backward:
0 E P C X
```

E

Source Code

```
82  template <typename T>
83  void dllDelNode(T findData, DoubleList<T>** currentHead) {
84      if(currentHead == nullptr || *currentHead == nullptr) return;
85
86      DoubleList<T>* currNode = *currentHead;
87
88      while (currNode != nullptr && currNode->data != findData) {
89          currNode = currNode->next;
90      }
91
92      if (currNode == nullptr) return;
93
94      if (currNode == *currentHead) {
95          *currentHead = currNode->next;
96      }
97
98      if (currNode->prev != nullptr) {
99          currNode->prev->next = currNode->next;
100     }
101
102     if (currNode->next != nullptr) {
103         currNode->next->prev = currNode->prev;
104     }
105
106     delete currNode;
107 }
```

	Console	Forward X C 6 E 0 Backward: 0 E 6 C X

F	Source Code	109 template <typename T>
		110 void dllDeleteAllNodes(DoubleList<T> **head){
		111 DoubleList<T>* current = *head;
		112
		113 while (current != nullptr) {
		114 DoubleList<T>* temp = current;
		115 current = current->next;
		116 delete temp;
		117 }
		118
		119 *head = nullptr;
		120 }

	Console	Deleting All Nodes The list is empty

Screenshots	Analysis

```
82  template <typename T>
83  void dllDelNode(T findData, DoubleList<T>** currentHead) {
84      if(currentHead == nullptr || *currentHead == nullptr) return;
85
86      DoubleList<T>* currNode = *currentHead;
87
88      while (currNode != nullptr && currNode->data != findData) {
89          currNode = currNode->next;
90      }
91
92      if (currNode == nullptr) return;
93
94      if (currNode == *currentHead) {
95          *currentHead = currNode->next;
96      }
97
98      if (currNode->prev != nullptr) {
99          currNode->prev->next = currNode->next;
100     }
101
102     if (currNode->next != nullptr) {
103         currNode->next->prev = currNode->prev;
104     }
105
106     delete currNode;
107 }
```

The dllDelNode function safely searches for a node matching a target value in a Doubly Linked List, updates the surrounding forward and backward pointers to maintain list integrity, and deallocates the target node's heap memory in $\mathcal{O}(N)$time.

```
template <typename T>
void dllDeleteAllNodes(DoubleList<T> **h
    DoubleList<T>* current = *head;

    while (current != nullptr) {
        DoubleList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
}
```

The dllDeleteAllNodes function traverses the entire Doubly Linked List sequentially to deallocate each node's memory via a temporary pointer, completely clearing the list and setting the primary head pointer to nullptr in $\mathcal{O}(N)$ time to prevent memory leaks and dangling references.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code:

supplementary.cpp > main()

```cpp
#include <iostream>
#include <string>

struct SongNode {
    std::string title;
    SongNode* next;
    SongNode* prev;

    SongNode(std::string name) {
        title = name;
        next = NULL;
        prev = NULL;
    }
};

class Playlist {
private:
    SongNode* head;
    SongNode* current;

public:
    Playlist() {
        head = NULL;
        current = NULL;
    }

    void addSong(std::string title) {
        SongNode* newSong = new SongNode(title);
```

```
        if (head == NULL) {
            head = newSong;
            head->next = head;
            head->prev = head;
            current = head;
            std::cout << "\"" << title << "\" added as the first song.\n";
            return;
        }

        SongNode* tail = head->prev;

        tail->next = newSong;
        newSong->prev = tail;
        newSong->next = head;
        head->prev = newSong;

        std::cout << "\"" << title << "\" added to the playlist.\n";
    }

    void removeSong(std::string title) {
        if (head == NULL) {
            std::cout << "Playlist is empty! Cannot remove.\n";
            return;
        }

        SongNode* temp = head;
        bool found = false;

        do {
            if (temp->title == title) {
                found = true;
                break;
            }
            temp = temp->next;
        } while (temp != head);

        if (!found) {
            std::cout << "Song \"" << title << "\" not found in playlist.\n";
            return;
        }
```

```
        if (temp->next == temp) {
            delete temp;
            head = NULL;
            current = NULL;
            std::cout << "Removed \"" << title << "\". Playlist is now empty.\n";
            return;
        }

        if (temp == head) {
            head = head->next;
        }

        if (temp == current) {
            current = current->next;
        }
```

```
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;

    delete temp;
    std::cout << "Successfully removed \"" << title << "\".\n";
}

void playNext() {
    if (current == NULL) {
        std::cout << "Playlist is empty.\n";
        return;
    }
    current = current->next;
    std::cout << "Now Playing: " << current->title << "\n";
}
```

```
    void playPrev() {
        if (current == NULL) {
            std::cout << "Playlist is empty.\n";
            return;
        }
        current = current->prev;
        std::cout << "Now Playing: " << current->title << "\n";
    }

    void playAll() {
        if (head == NULL) {
            std::cout << "Playlist is empty.\n";
            return;
        }
```

```
std::cout << "\n--- Playlist Songs ---\n";
SongNode* temp = head;
int trackNum = 1;

do {
    std::cout << trackNum << ". " << temp->title;
    if (temp == current) {
        std::cout << "  <-- [Currently Selected]";
    }
    std::cout << "\n";
    temp = temp->next;
    trackNum++;
} while (temp != head);
```

```
135
136 int main() {
137     Playlist myPlaylist;
138     int choice;
139     std::string songTitle;
140
141     myPlaylist.addSong("Take it to the Limit");
142     myPlaylist.addSong("Open Arms");
143     myPlaylist.addSong("Don't Stop Belivin'");
144
145     while (true) {
146         std::cout << "\n==============================\n";
147         std::cout << "   VibeCheck  \n";
148         std::cout << "==============================\n";
149         std::cout << "1. Add Song\n";
150         std::cout << "2. Remove Song\n";
151         std::cout << "3. Next Song\n";
152         std::cout << "4. Previous Song\n";
153         std::cout << "5. Display All Songs\n";
154         std::cout << "6. Exit\n";
155         std::cout << "Enter choice: ";
156         std::cin >> choice;
157         std::cin.ignore();
158
159         switch (choice) {
160             case 1:
161                 std::cout << "Enter song title: ";
162                 std::getline(std::cin, songTitle);
163                 myPlaylist.addSong(songTitle);
164                 break;
165             case 2:
166                 std::cout << "Enter song title to remove: ";
167                 std::getline(std::cin, songTitle);
168                 myPlaylist.removeSong(songTitle);
```

```
169                 break;
170             case 3:
171                 myPlaylist.playNext();
172                 break;
173             case 4:
174                 myPlaylist.playPrev();
175                 break;
176             case 5:
177                 myPlaylist.playAll();
178                 break;
179             case 6:
180                 std::cout << "Exiting music player...\n";
181                 return 0;
182             default:
183                 std::cout << "Invalid choice! Try again.\n";
184         }
185     }
186
```

```
184         }
185     }
186
187     return 0;
188 }
```

Output:

```
"Take it to the Limit" added as the first song.
"Open Arms" added to the playlist.
"Don't Stop Belivin'" added to the playlist.

==============================
   VibeCheck
==============================
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 1
Enter song title: Faithfully
"Faithfully" added to the playlist.

==============================
   VibeCheck
==============================
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 6
Exiting music player...
```

Conclusion:
This activity successfully demonstrated the implementation and practical application of Doubly Linked Lists and circular doubly linked lists in C++. By executing node insertion, deletion and bidirectional traversal, the core principles of dynamic memory allocation and precise pointer management were effectively reinforced. Real world applications such as a continuous music playlist further highlighted the efficiency of circular doubly linked structures for dynamic data handling and seamless navigation.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, code assistance and refining the code**

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was utilized to help explain core linked list concepts and assist in debugging and refining C++ code structure. The AI was used solely for supportive guidance and did not replace my independent problem solving, code implementation, analysis or primary authorship of this report.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.